DALHOUSIE UNIVERSITY

CSCI 5408: Data Management, Warehousing and Analytics

Assignment: 01

Banner ID: B01026328

Name: Parva Mineshbhai Patel

Date: 2025 – 03 – 07

GitLab Assignment Link: https://git.cs.dal.ca/parva/
CSCI5408_W25_B01026328_ParvaMineshbhai_Patel

Contents

1) Problem 1 (LO#1): Explore the provided web links and build an ERD for the given scenario…..1
2) Initial ER Diagram……3
3) Final ER Diagram……4
4) EERD Diagram……5
5) Problem 2 (LO#2): Prototype a lightweight DBMS in Java, as described in the table below……6
6) Task A1 ……7
7) Task A2……9
8) Task A3 and Test Cases ……11
9) Task B1……13
10) Task B2……15
11) Task B3……16
12) Task C1……17
13) Task C2……20
14) Test Cases……23
15) References……29

1) **Problem 1 (LO#1): Explore the provided web links and build an ERD for the given scenario**

Research Table

Entities	**Attributes**
City	city_id (PK), name, province
Neighborhood	neighborhood_id (PK), name, area
Park	park_id (PK), name, location, area, population
ParkNeighborhood	park_neighborhood_id (PK), usage_type, population
School	school_id (PK), name, address, level, capacity, area
SchoolNeighborhood	school_neighborhood_id (PK), funding_source, student_count, service_radius
HealthFacility	facility_id (PK), name, address, type
FacilityServiceArea	service_area_id (PK), service_radius, specialties
UtilityService	service_id (PK), type, provider_name, contact_info
BuildingPermit	permit_id (PK), type, issue_date, expiration_date, location, address
PublicFacility	facility_id (PK), name, type
PublicArtInstallation	installation_id (PK), title, location, installation_date
TouristAttraction	attraction_id (PK), name, type, location
Business	business_id (PK), name, industry, address
Citizen	citizen_id (PK), name, contact_info, address
CitizenComplaint	complaint_id (PK), type, date, description
CrimeIncident	incident_id (PK), type, date, location
EmergencyServices	service_id (PK), type, contact_info
TrafficSignal	signal_id (PK), location, installation_date, status
Intersection	intersection_id (PK), location, road_names, traffic_volume
PublicEvent	event_id (PK), name, date, location
ParkingFacility	parking_id (PK), type, capacity, location
WasteCollection	collection_id (PK), type, schedule, area_served
EnvironmentalSensor	sensor_id (PK), type, location, status
PublicTransitStop	stop_id (PK), name, location
PublicTransitRoute	route_id (PK), route_name, start_location, end_location, total_distance
RouteStop	routestop_id (PK), arrival_time, departure_time
PublicTransitSchedule	schedule_id (PK), frequency, departure_time, arrival_time

- **City** – Represents the main administrative unit for all urban infrastructure.
- **Neighborhood** – Defines subdivisions of a city to manage localized data.
- **Park** – Tracks public green spaces available to citizens.
- **ParkNeighborhood** – Links parks to neighborhoods they serve.
- **School** – Represents educational institutions within the city.
- **SchoolNeighborhood** – Connects schools to the neighborhoods they serve.
- **HealthFacility** – Manages hospitals, clinics, and other medical facilities.
- **FacilityServiceArea** – Defines the service coverage of health facilities.
- **UtilityService** – Stores details about essential city utilities (e.g., water, electricity).
- **BuildingPermit** – Tracks construction permits issued within the city.
- **PublicFacility** – Represents public-use buildings like libraries and community centers.
- **PublicArtInstallation** – Records artistic structures installed in public spaces.
- **TouristAttraction** – Stores information about places of interest for visitors.
- **Business** – Manages commercial entities operating in the city.
- **Citizen** – Represents individual residents interacting with city services.
- **CitizenComplaint** – Tracks complaints filed by citizens regarding urban services.
- **CrimeIncident** – Records crime occurrences within the city for public safety monitoring.
- **EmergencyServices** – Stores emergency response units like police and fire stations.
- **TrafficSignal** – Manages the placement and operation of traffic signals.
- **PublicEvent** – Tracks city-organized or permitted public events.

2) Initial ER Diagram

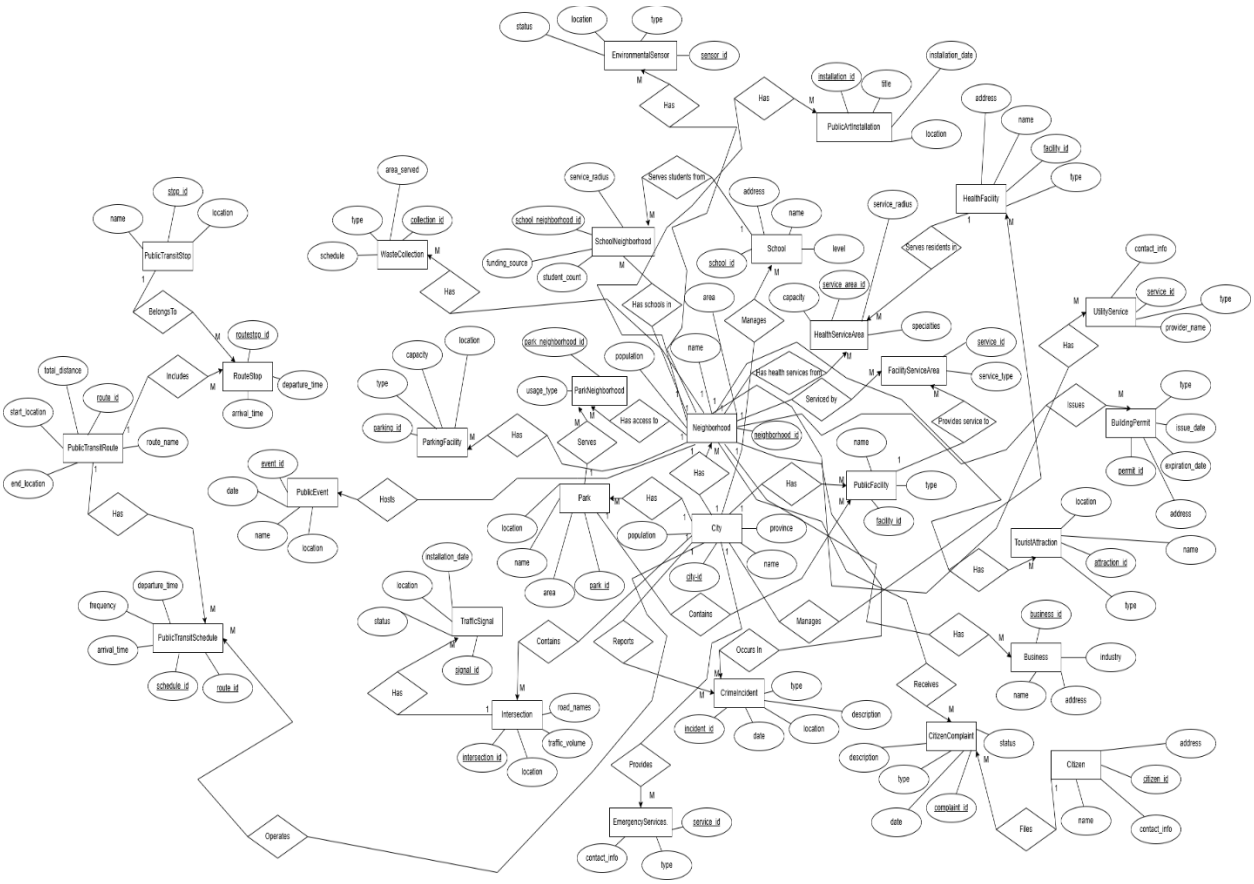


Figure 1

Design Issues

1. Fixing Fan Traps

Issue: PublicFacility is directly linked to City.
Fix:

- Remove the **direct City → PublicFacility relationship.**
- **PublicFacility is connected via Neighborhood.**
- Final structure: City → (Has) → Neighborhood → (Has) → PublicFacility

Issue: ParkNeighborhood must mediate between Neighborhood and Park.
Fix:

- **ParkNeighborhood mediates** the relationship between Neighborhood and Park.

- Avoid direct multiple paths from **City to Park** that bypass Neighborhood.
- Final structure: City → (Has) → Neighborhood → (Serves) → ParkNeighborhood → (Has Access to) → Park

2. Fixing Chasm Traps

Issue: HealthFacility is not properly connected to FacilityServiceArea.
Fix:

- Remove direct connection from **City → HealthFacility**
- **HealthFacility is linked through FacilityServiceArea.**
- Final structure: City → (Manages) → FacilityServiceArea → (Serviced by) → HealthFacility

3. Fixing Overlapping Relationships

Issue: PublicFacility is redundantly linked to City and Neighborhood.
Fix:

- Remove the **direct City → PublicFacility link**
- **PublicFacility is only linked via Neighborhood**
- Final structure: City → Neighborhood → PublicFacility

4. Fixing General Complexity Issues

Issue: PublicTransitRoute → RouteStop → PublicTransitStop structured properly.
Fix:

- **RouteStop is an associative entity** between PublicTransitRoute and PublicTransitStop.
- Final structure: PublicTransitRoute → (Includes) → RouteStop → (Belongs to) → PublicTransitStop

Issue: Business and TouristAttraction not have redundant connections.
Fix:

- Remove direct Business → City if already connected via TouristAttraction
- **TouristAttraction only links to Business when necessary**
- Final structure: City → Neighborhood → Business → TouristAttraction (If applicable)

3) Final ER Diagram

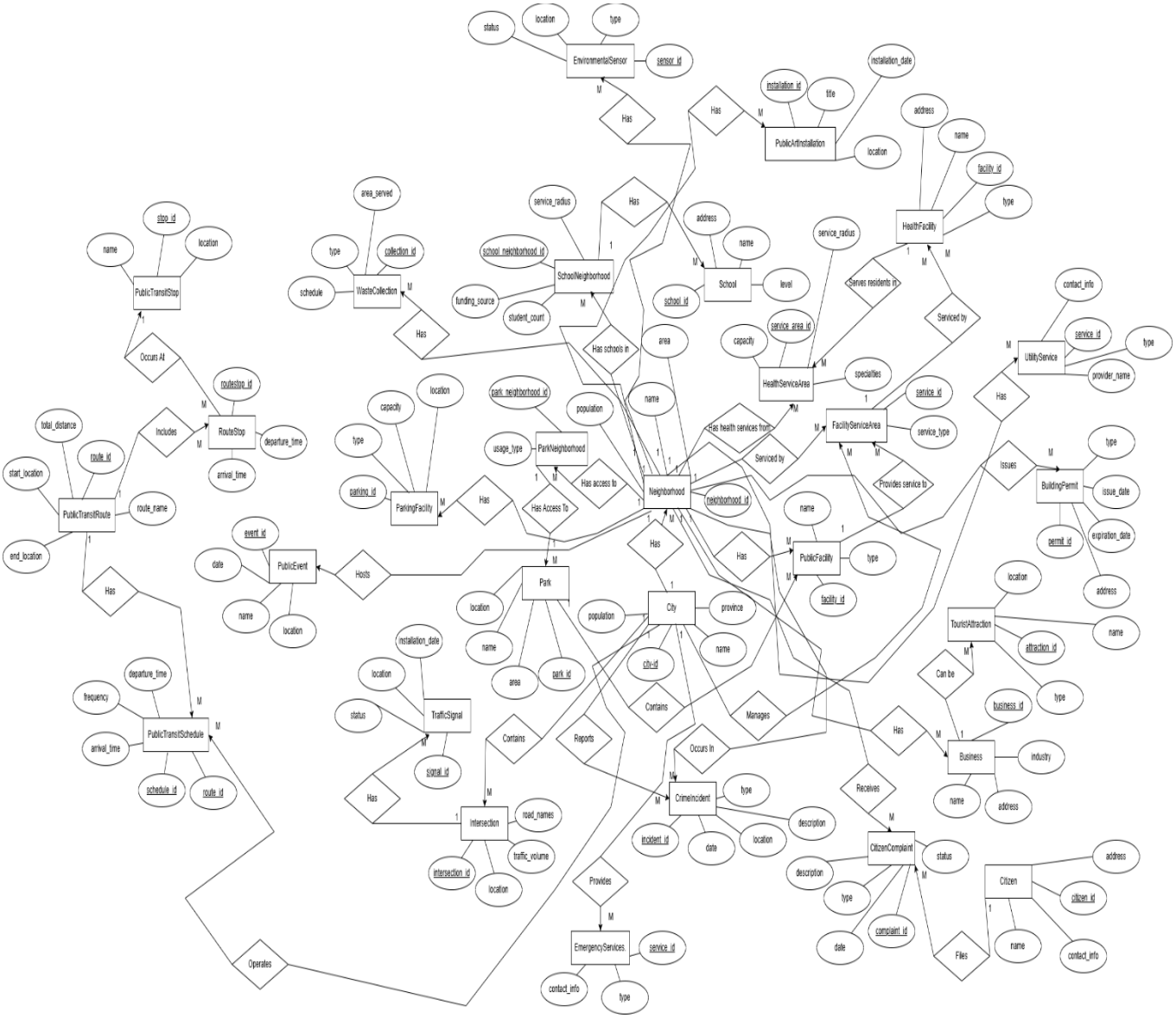


Figure 2

4) EERD Diagram

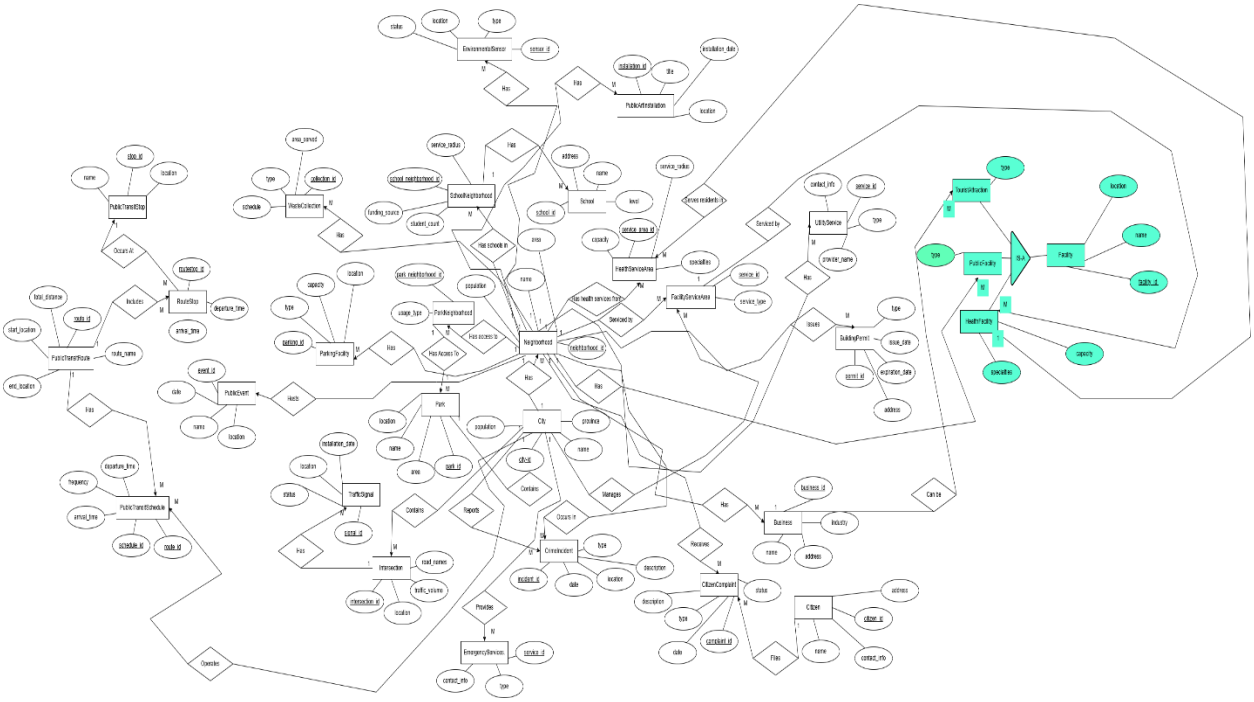


Figure 3

Specialization applied to the Facility entity by categorizing it into three specific subtypes:

1. **HealthFacility**
2. **TouristAttraction**
3. **PublicFacility**

Reason for Specialization:

- All three entities share **common attributes** (facility_id, name, location).
- Specialization allows the **specific attributes** of each facility type to be represented without redundancy.
- Using an **ISA relationship**, each facility type inherits the **common attributes** while maintaining its unique characteristics.

5) **Problem 2 (LO#2): Prototype a lightweight DBMS in Java, as described in the table below.**

S.O.L.I.D Principles

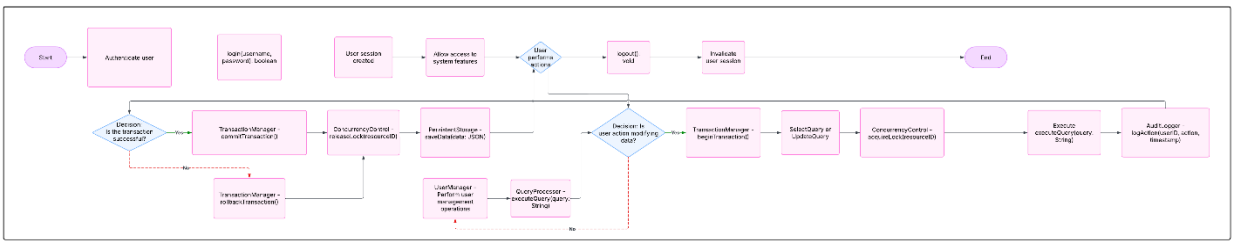


Figure 4

1. Single Responsibility Principle (SRP)

Each class have only one reason to change:

- Authentication.java → Handles user authentication.
- UserManager.java → Manages user-related operations.
- Query.java → Parses and executes user queries.
- PersistentStorage.java → Manages storage operations (reading/writing from database.json).
- AuditLogger.java → Logs audit information in audit_log.json.

2. Open/Closed Principle (OCP)

Design allow extension without modification:

- Implement interfaces for storage (StorageInterface for PersistentStorage.java and InMemoryIndex.java).
- Use abstract classes to define behavior that subclasses can extend (e.g., QueryProcessor extended by SelectQuery, UpdateQuery).

3. Liskov Substitution Principle (LSP)

Subtypes should be replaceable by base types:

- PersistentStorage and InMemoryIndex implement a common interface (StorageInterface).
- TransactionManager.java operate on StorageInterface, allowing flexibility in storage mechanisms.

4. Interface Segregation Principle (ISP)

Clients not be forced to depend on interfaces they do not use:

- Instead of a single DatabaseSystem.java handling all database operations, create smaller interfaces:
    - TransactionManager (for TransactionManager.java)
    - Query (for Query.java)
    - AuditLogger (for AuditLogger.java)

5. Dependency Inversion Principle (DIP)

High-level modules not depend on low-level modules, but on abstractions:

- Use dependency injection for AuditLogger.java, Persist

6) Task A1

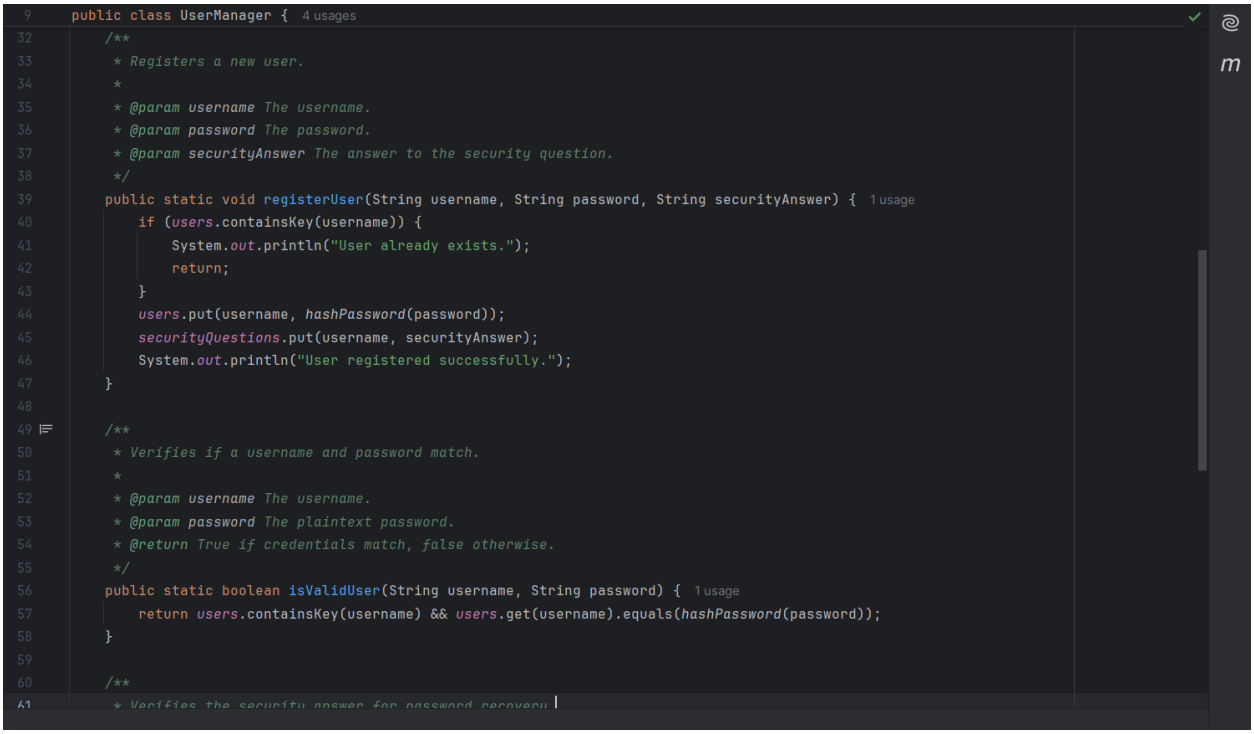
Implementation of User Manager

The UserManager class is responsible for managing user accounts, including registering new users and verifying their credentials during login.

```java
package org.example;

import java.security.*;
import java.util.*;

/**
 * Manages user credentials, security questions, and password hashing.
 */
public class UserManager {  4 usages
    private static final Map<String, String> users = new HashMap<>();  5 usages
    private static final Map<String, String> securityQuestions = new HashMap<>();  2 usages
    /**
     * Hashes a password using SHA-256.
     *
     * @param password The plaintext password.
     * @return The hashed password.
     */
    public static String hashPassword(String password) {  3 usages
        try {
            MessageDigest digest = MessageDigest.getInstance( algorithm: "SHA-256");
            byte[] hash = digest.digest(password.getBytes());
            StringBuilder hexString = new StringBuilder();
            for (byte b : hash) {
                hexString.append(String.format("%02x", b));
            }
            return hexString.toString();
        } catch (NoSuchAlgorithmException e) {
            throw new RuntimeException("Error hashing password.", e);
        }
    }
```

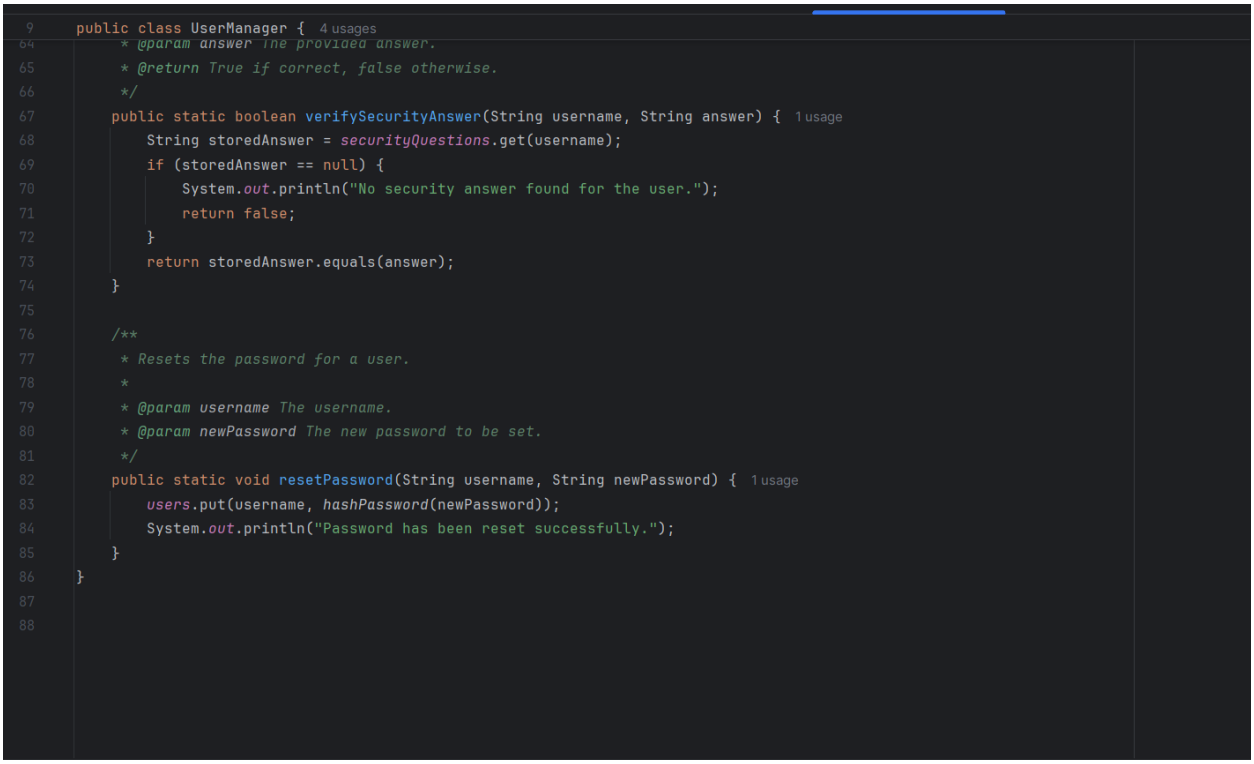
Figure 5

It stores user information such as usernames, passwords, and security question answers, ensuring that only authorized users can access the system.



```
public class UserManager {  4 usages
    /**
     * Registers a new user.
     *
     * @param username The username.
     * @param password The password.
     * @param securityAnswer The answer to the security question.
     */
    public static void registerUser(String username, String password, String securityAnswer) {  1 usage
        if (users.containsKey(username)) {
            System.out.println("User already exists.");
            return;
        }
        users.put(username, hashPassword(password));
        securityQuestions.put(username, securityAnswer);
        System.out.println("User registered successfully.");
    }

    /**
     * Verifies if a username and password match.
     *
     * @param username The username.
     * @param password The plaintext password.
     * @return True if credentials match, false otherwise.
     */
    public static boolean isValidUser(String username, String password) {  1 usage
        return users.containsKey(username) && users.get(username).equals(hashPassword(password));
    }

    /**
     * Verifies the security answer for password recovery
```

Figure 6



```
public class UserManager {  4 usages
     * @param answer The provided answer.
     * @return True if correct, false otherwise.
     */
    public static boolean verifySecurityAnswer(String username, String answer) {  1 usage
        String storedAnswer = securityQuestions.get(username);
        if (storedAnswer == null) {
            System.out.println("No security answer found for the user.");
            return false;
        }
        return storedAnswer.equals(answer);
    }

    /**
     * Resets the password for a user.
     *
     * @param username The username.
     * @param newPassword The new password to be set.
     */
    public static void resetPassword(String username, String newPassword) {  1 usage
        users.put(username, hashPassword(newPassword));
        System.out.println("Password has been reset successfully.");
    }
}
```

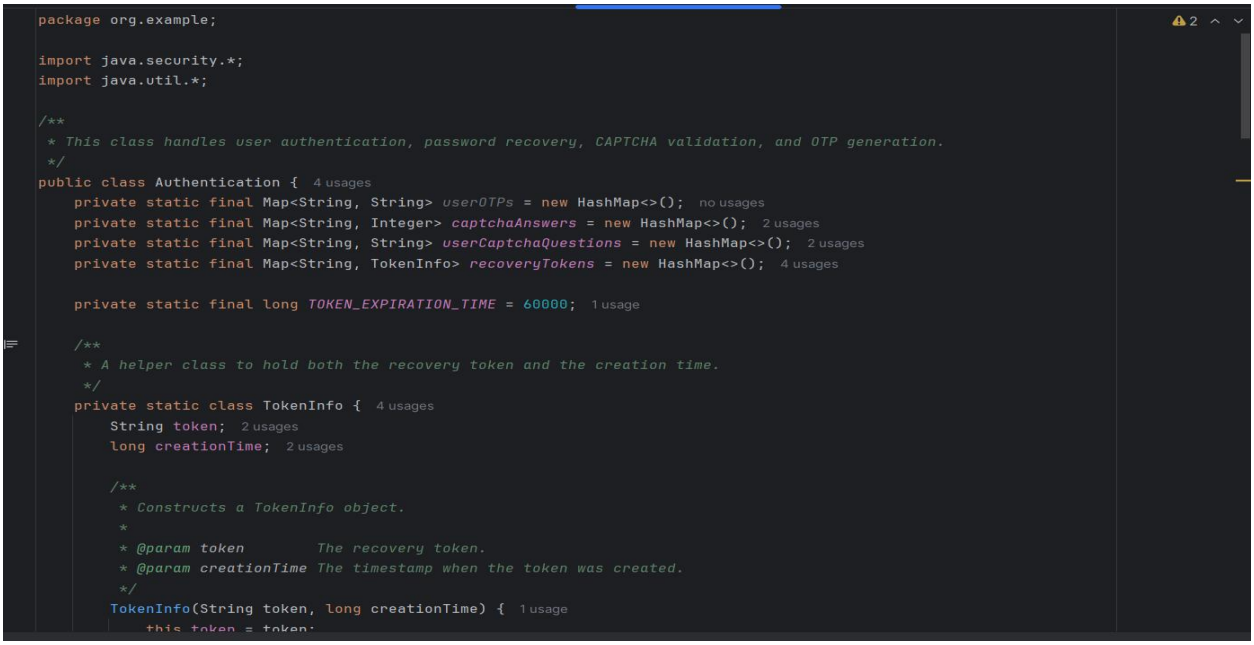
Figure 7

The class also provides functionality to reset passwords by verifying security question answers and handling user-related operations securely.

7) Task A2

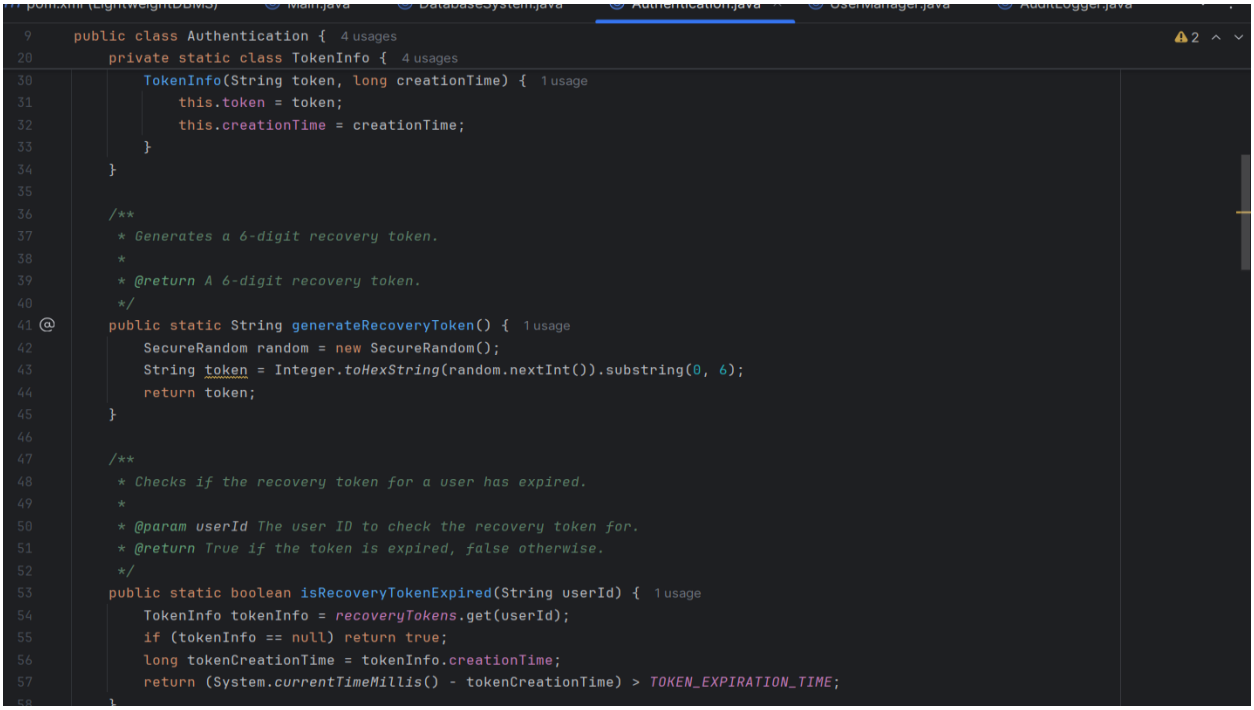
Implementation of Authentication

The Authentication class implements user signup and login functionality, allowing users to register with a username, password, and security question.



```java
package org.example;

import java.security.*;
import java.util.*;

/**
 * This class handles user authentication, password recovery, CAPTCHA validation, and OTP generation.
 */
public class Authentication {  4 usages
    private static final Map<String, String> userOTPs = new HashMap<>();  no usages
    private static final Map<String, Integer> captchaAnswers = new HashMap<>();  2 usages
    private static final Map<String, String> userCaptchaQuestions = new HashMap<>();  2 usages
    private static final Map<String, TokenInfo> recoveryTokens = new HashMap<>();  4 usages

    private static final long TOKEN_EXPIRATION_TIME = 60000;  1 usage

    /**
     * A helper class to hold both the recovery token and the creation time.
     */
    private static class TokenInfo {  4 usages
        String token;  2 usages
        long creationTime;  2 usages

        /**
         * Constructs a TokenInfo object.
         *
         * @param token        The recovery token.
         * @param creationTime The timestamp when the token was created.
         */
        TokenInfo(String token, long creationTime) {  1 usage
            this.token = token;
```

Figure 8



```java
public class Authentication {  4 usages
    private static class TokenInfo {  4 usages
        TokenInfo(String token, long creationTime) {  1 usage
            this.token = token;
            this.creationTime = creationTime;
        }
    }

    /**
     * Generates a 6-digit recovery token.
     *
     * @return A 6-digit recovery token.
     */
    public static String generateRecoveryToken() {  1 usage
        SecureRandom random = new SecureRandom();
        String token = Integer.toHexString(random.nextInt()).substring(0, 6);
        return token;
    }

    /**
     * Checks if the recovery token for a user has expired.
     *
     * @param userId The user ID to check the recovery token for.
     * @return True if the token is expired, false otherwise.
     */
    public static boolean isRecoveryTokenExpired(String userId) {  1 usage
        TokenInfo tokenInfo = recoveryTokens.get(userId);
        if (tokenInfo == null) return true;
        long tokenCreationTime = tokenInfo.creationTime;
        return (System.currentTimeMillis() - tokenCreationTime) > TOKEN_EXPIRATION_TIME;
    }
```

Figure 9

It also handles password recovery through secure tokens, ensuring that the user can reset their password with a temporary recovery token.

The class integrates CAPTCHA verification to prevent automated login attempts and enforces token expiration to enhance security.

```
public class Authentication {  4 usages
    */
    public static boolean verifyAndResetPassword(String userId, String token, String newPassword) {  1 usage
        TokenInfo tokenInfo = recoveryTokens.get(userId);
        if (tokenInfo == null || !tokenInfo.token.equals(token)) {
            System.out.println("Invalid or expired recovery token.");
            return false;
        }

        if (isRecoveryTokenExpired(userId)) {
            System.out.println("Token expired! You can no longer reset your password with this token.");
            return false;
        }

        UserManager.resetPassword(userId, newPassword);
        recoveryTokens.remove(userId); // Remove the token after successful reset
        return true;
    }
```

Figure 10

```
 * @return The recovery token if successful, null if the security answer is incorrect.
 */
public static String passwordRecovery(String userId, String answer) {  1 usage
    if (!UserManager.verifySecurityAnswer(userId, answer)) {
        System.out.println("Incorrect Security Answer.");
        return null;
    }

    String recoveryToken = generateRecoveryToken();
    recoveryTokens.put(userId, new TokenInfo(recoveryToken, System.currentTimeMillis()));
    System.out.println("Recovery Token: " + recoveryToken);
    System.out.println("Your token expires in 1 minute.");
    return recoveryToken;
}
```

Figure 11

```
    public static boolean verifyCaptcha(String userId, String userInput) {  1 usage
        Integer correctAnswer = captchaAnswers.get(userId);
        if (correctAnswer == null) {
            System.out.println("CAPTCHA expired or not initialized.");
            return false;
        }

        return correctAnswer.toString().equals(userInput);
    }
```

Figure 12

Additionally, it provides methods for password verification and account authentication, ensuring that only valid users can access the system.

8) Task A3

Implementation of Audit Logger

The AuditLogger class is responsible for logging user events such as login attempts and password changes to ensure a secure and auditable trail of actions.

```
 * Handles logging of authentication events.
 */
public class AuditLogger {  2 usages
    private static final List<String> logs = new ArrayList<>();  2 usages

    /**
     * Logs authentication attempts.
     *
     * @param userId The user ID.
     * @param success True if login was successful, false otherwise.
     */
    public static void logEvent(String userId, boolean success) {  2 usages
        String timestamp = new Date().toString();
        String ipAddress = "192.168.1." + new Random().nextInt( bound: 255);
        String logEntry = String.format("Timestamp: %s | User: %s | Success: %b | IP: %s", timestamp, userId, success, ipAddress);

        logs.add(logEntry);
        saveLogsToFile();
    }

    /**
     * Saves logs to a file in JSON format.
     */
    private static void saveLogsToFile() {  1 usage
        try (FileWriter file = new FileWriter( fileName: "audit_log.json")) {
            file.write( str: "{ \"logs\": " + logs.toString() + " }");
        } catch (IOException e) {
            System.out.println("Error writing logs to file.");
        }
    }
```

Figure 13

It records both successful and failed authentication attempts, helping track suspicious activities and ensure compliance with security policies. The class ensures that every significant event is logged with the appropriate timestamp and user details, providing transparency and accountability for system access.

Test Cases for Authentication

1) **Register Successful:** When the user is new, they just enter their username and password and also set the security question answer.

Figure 14

2) **Login Successful:** When the user enter their valid username, password and write correct Captcha so they will login successfully.

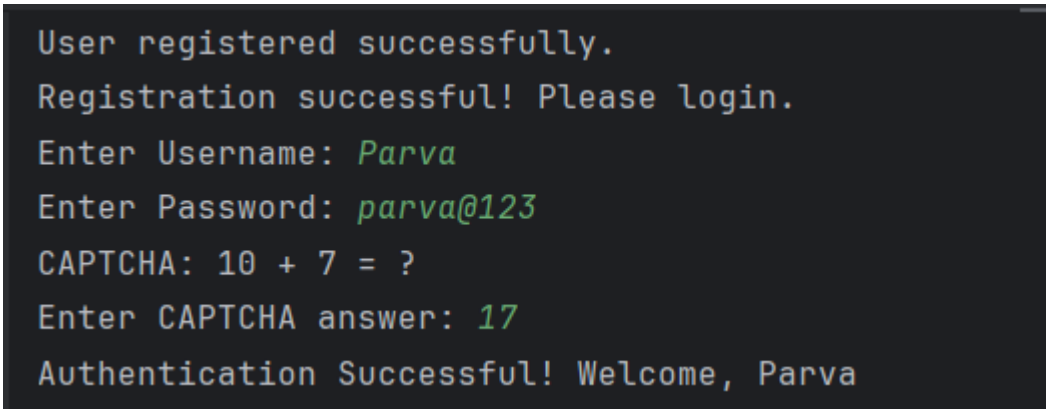


Figure 15

3) **Login Failed**: When the user writes incorrect password it will show the error like Incorrect Password.

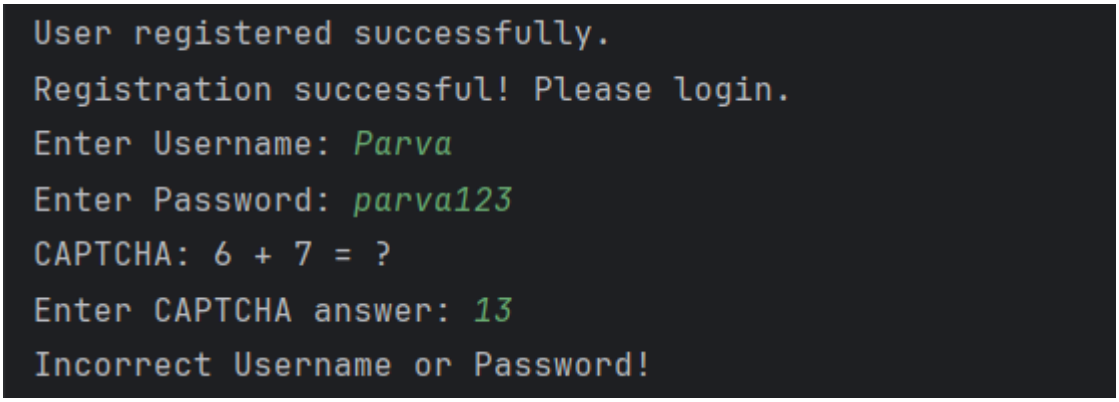


Figure 16

4) **Reset Password Successful** – The user wants to reset their password so they must write their security answer and also the recovery token within one minute because after that it will expire.

Figure 17

5) Login Failed for incorrect user: When user enters wrong username it will shows error Incorrect username.

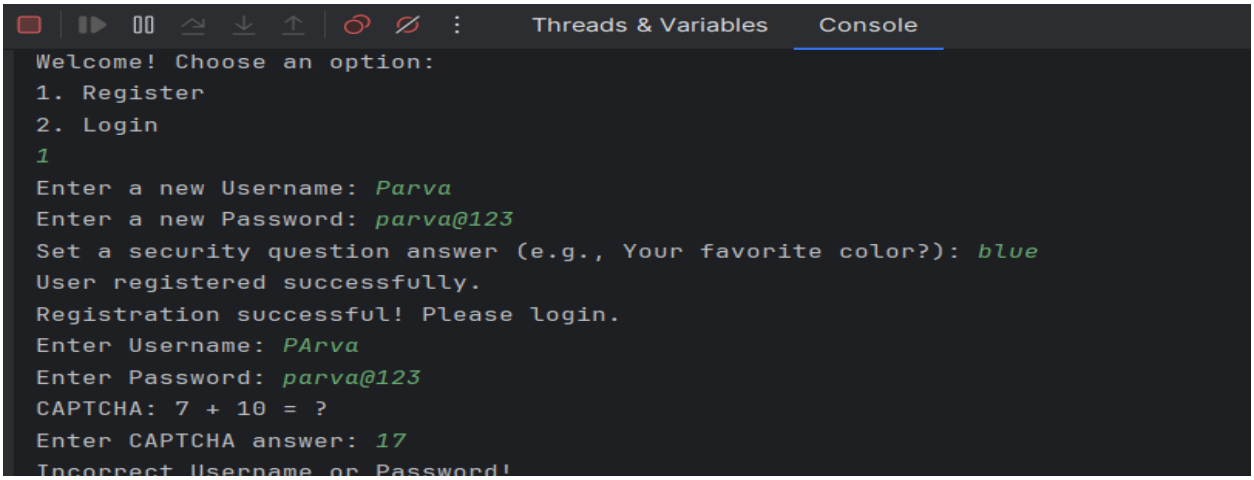


Figure 18

9) Audit_log.json

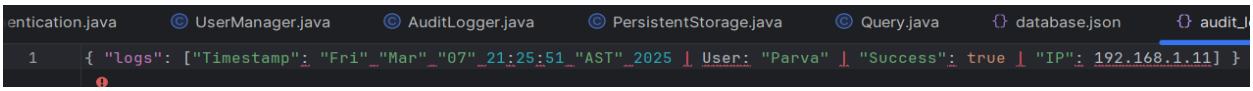
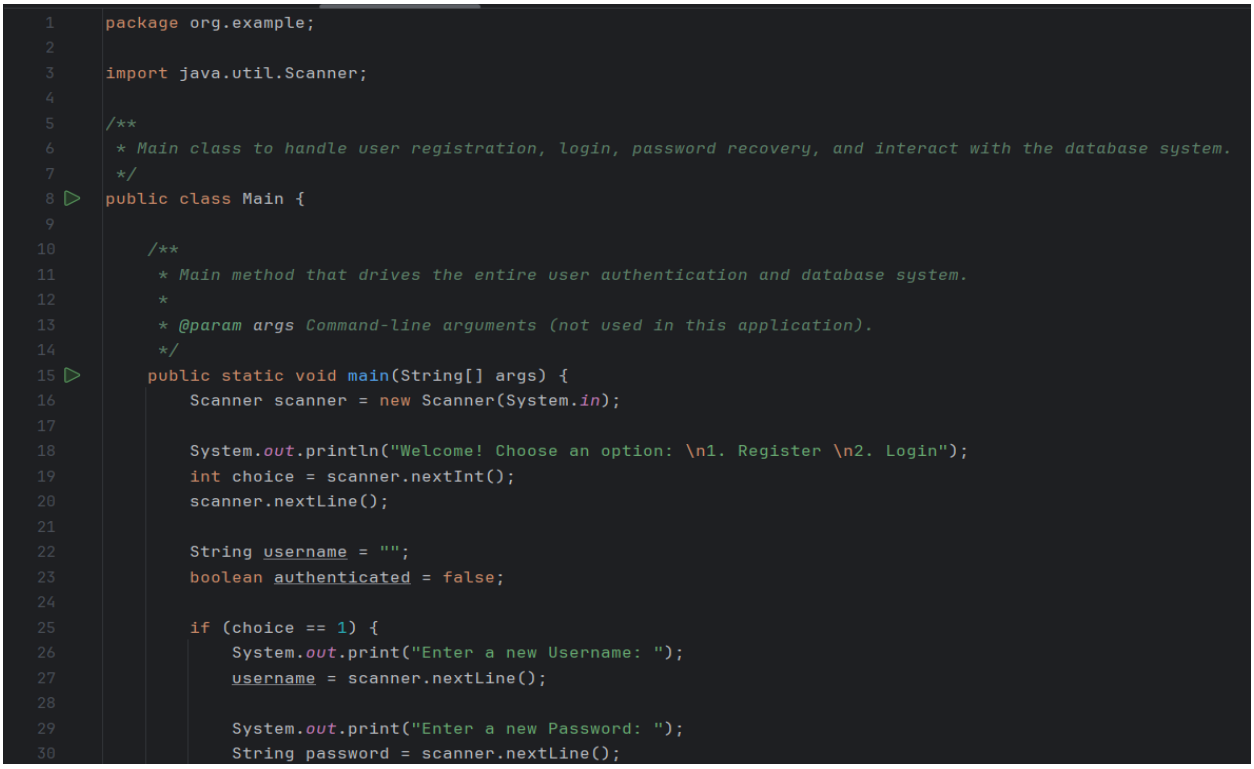


Figure 19

Main



```
1   package org.example;
2
3   import java.util.Scanner;
4
5   /**
6    * Main class to handle user registration, login, password recovery, and interact with the database system.
7    */
8   public class Main {
9
10      /**
11       * Main method that drives the entire user authentication and database system.
12       *
13       * @param args Command-line arguments (not used in this application).
14       */
15      public static void main(String[] args) {
16          Scanner scanner = new Scanner(System.in);
17
18          System.out.println("Welcome! Choose an option: \n1. Register \n2. Login");
19          int choice = scanner.nextInt();
20          scanner.nextLine();
21
22          String username = "";
23          boolean authenticated = false;
24
25          if (choice == 1) {
26              System.out.print("Enter a new Username: ");
27              username = scanner.nextLine();
28
29              System.out.print("Enter a new Password: ");
30              String password = scanner.nextLine();
```

Figure 20

```
public class Main {
    public static void main(String[] args) {
            System.out.println("Registration successful! Please login.");
        }

        while (!authenticated) {
            System.out.print("Enter Username: ");
            username = scanner.nextLine();

            System.out.print("Enter Password: ");
            String password = scanner.nextLine();

            String captchaQuestion = Authentication.initiateCaptchaChallenge(username);
            System.out.println("CAPTCHA: " + captchaQuestion);

            System.out.print("Enter CAPTCHA answer: ");
            String captchaInput = scanner.nextLine();

            authenticated = Authentication.authenticate(username, password, captchaInput);
        }

        System.out.print("Forgot password? (y/n): ");
        String forgotPassword = scanner.nextLine();

        if (forgotPassword.equals("y")) {
            System.out.print("Answer your security question: ");
            String answer = scanner.nextLine();

            String token = Authentication.passwordRecovery(username, answer);
            if (token != null) {
```

Figure 21

pom.xml (LightweightDBMS) | Main.java | DatabaseSystem.java | Authentication.java | UserM

```
8     public class Main {
15        public static void main(String[] args) {
72                        }
73                    } else {
74                        System.out.println("Invalid recovery token.");
75                    }
76                }
77            }
78
79            System.out.println("Authentication successful! Starting Database System...");
80            DatabaseSystem dbSystem = new DatabaseSystem();
81            dbSystem.start();
82
83            while (true) {
84                System.out.println("Enter your SQL-like command (or type 'exit' to quit): ");
85                String command = scanner.nextLine();
86
87                if ("exit".equalsIgnoreCase(command)) {
88                    System.out.println("Exiting Database System...");
89                    break;
90                }
91
92
93                dbSystem.processCommand(command);
94            }
95
96            scanner.close();
97        }
98    }
```

Figure 22

10) Task B1

database.json

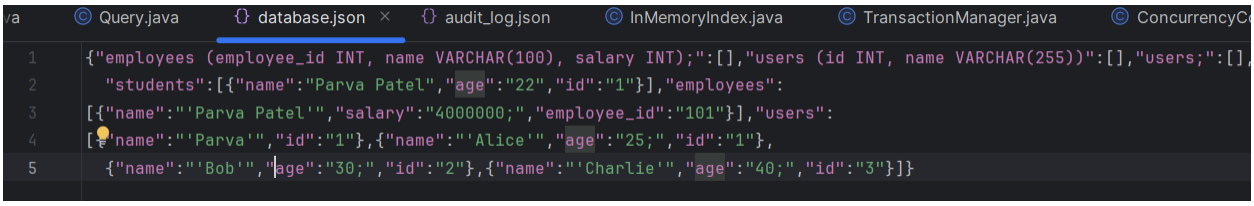


Figure 23

11) Task B2

Implementation of DatabaseSystem

The DatabaseSystem class serves as the core of the console-based database management system, handling user commands for database operations. It processes SQL-like queries such as creating databases, tables, inserting data, and retrieving records. The class ensures smooth interaction between the user and the database by validating commands and executing appropriate actions. It also manages transactions, providing support for BEGIN TRANSACTION, COMMIT, and ROLLBACK operations to maintain data integrity.

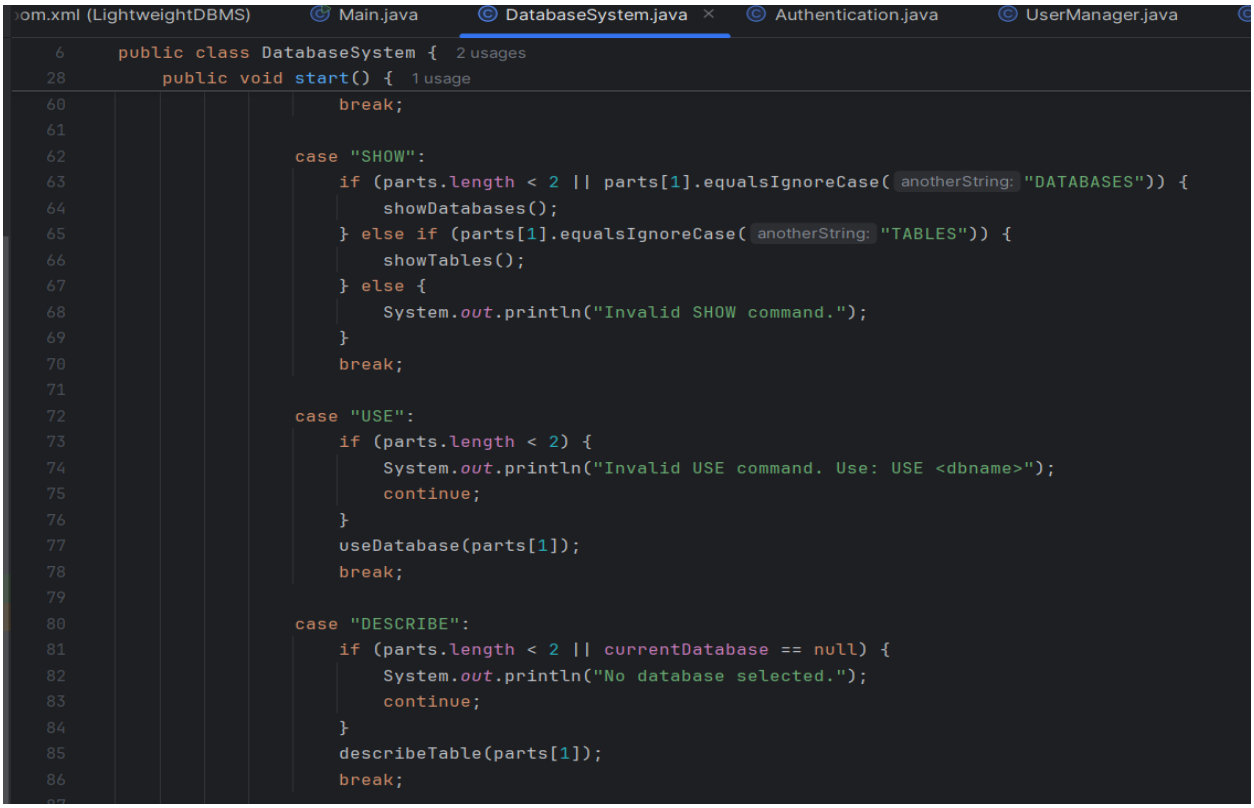


Figure 24

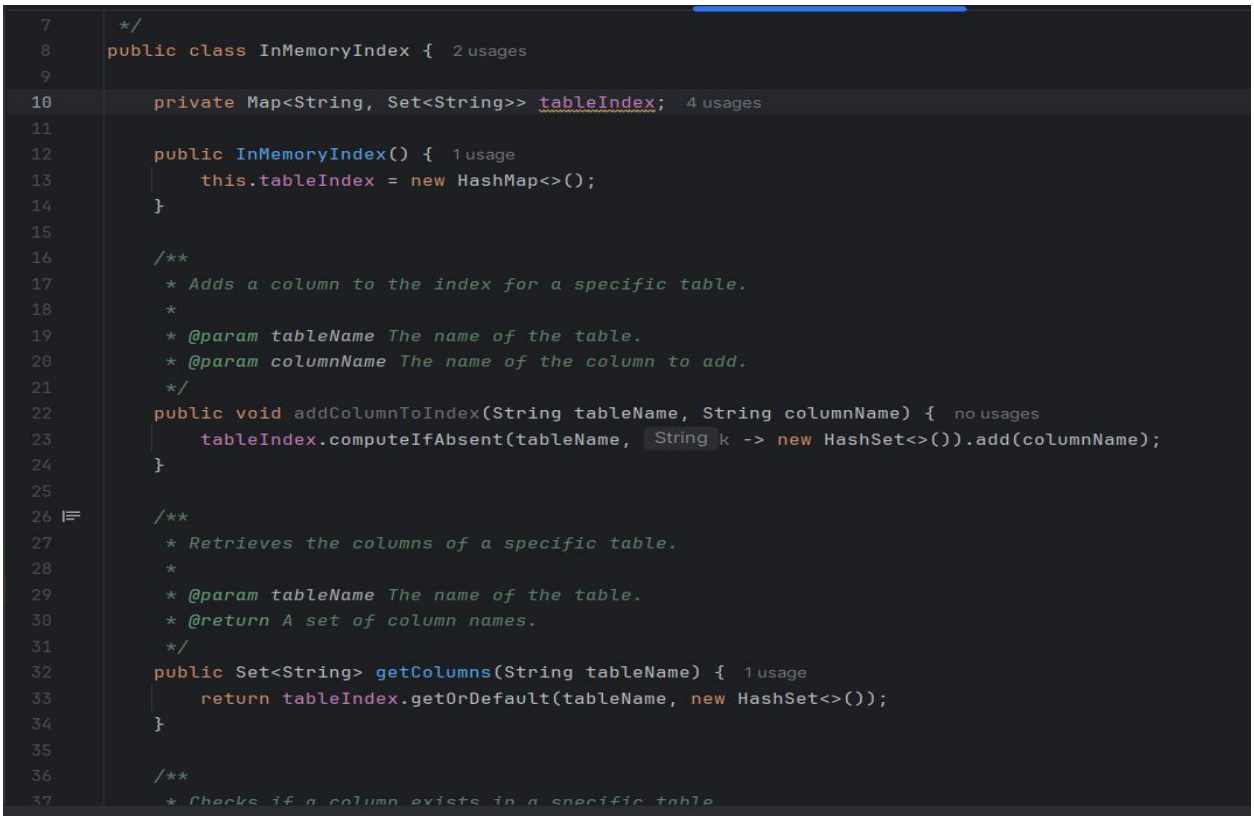
```
case "INSERT":
    if (parts.length < 3) {
        System.out.println("Invalid INSERT syntax. Use: INSERT INTO <table> VALUES (...).");
        continue;
    }
    processInsertCommand(parts[2]);
    break;

case "SELECT":
    if (parts.length < 2) {
        System.out.println("Invalid SELECT syntax. Use: SELECT * FROM <table>.");
        continue;
    }
    processSelectCommand(parts[1]);
    break;
```

Figure 25

12) Task B3

Implementation of InMemoryIndex

The InMemoryIndex class is responsible for managing indexing within the database system to optimize query performance.



```
 */
public class InMemoryIndex {  2 usages

    private Map<String, Set<String>> tableIndex;  4 usages

    public InMemoryIndex() {  1 usage
        this.tableIndex = new HashMap<>();
    }

    /**
     * Adds a column to the index for a specific table.
     *
     * @param tableName The name of the table.
     * @param columnName The name of the column to add.
     */
    public void addColumnToIndex(String tableName, String columnName) {  no usages
        tableIndex.computeIfAbsent(tableName, String k -> new HashSet<>()).add(columnName);
    }

    /**
     * Retrieves the columns of a specific table.
     *
     * @param tableName The name of the table.
     * @return A set of column names.
     */
    public Set<String> getColumns(String tableName) {  1 usage
        return tableIndex.getOrDefault(tableName, new HashSet<>());
    }

    /**
     * Checks if a column exists in a specific table
```

Figure 26

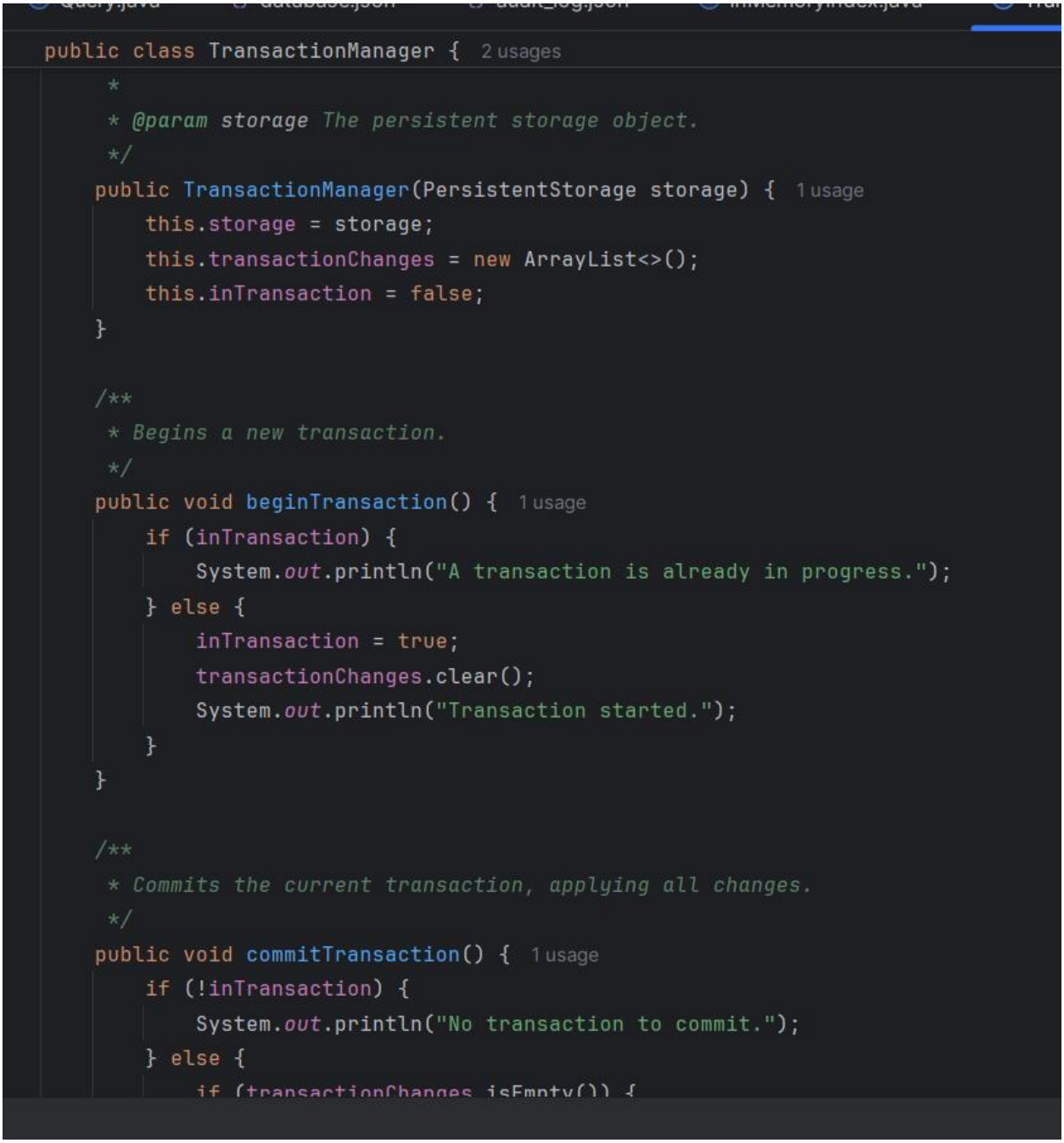
```java
 */
public class InMemoryIndex {  2 usages

    private Map<String, Set<String>> tableIndex;  4 usages

    public InMemoryIndex() {  1 usage
        this.tableIndex = new HashMap<>();
    }

    /**
     * Adds a column to the index for a specific table.
     *
     * @param tableName The name of the table.
     * @param columnName The name of the column to add.
     */
    public void addColumnToIndex(String tableName, String columnName) {  no usages
        tableIndex.computeIfAbsent(tableName, String k -> new HashSet<>()).add(columnName);
    }

    /**
     * Retrieves the columns of a specific table.
     *
     * @param tableName The name of the table.
     * @return A set of column names.
     */
    public Set<String> getColumns(String tableName) {  1 usage
        return tableIndex.getOrDefault(tableName, new HashSet<>());
    }

    /**
     * Checks if a column exists in a specific table
```

Figure 27

It stores indexed data in memory, allowing for faster lookups and retrieval of records based on specific attributes. This indexing mechanism helps improve search efficiency, particularly for SELECT queries, by reducing the need for full table scans. The class ensures that indexes are updated dynamically as new data is inserted, deleted, or modified

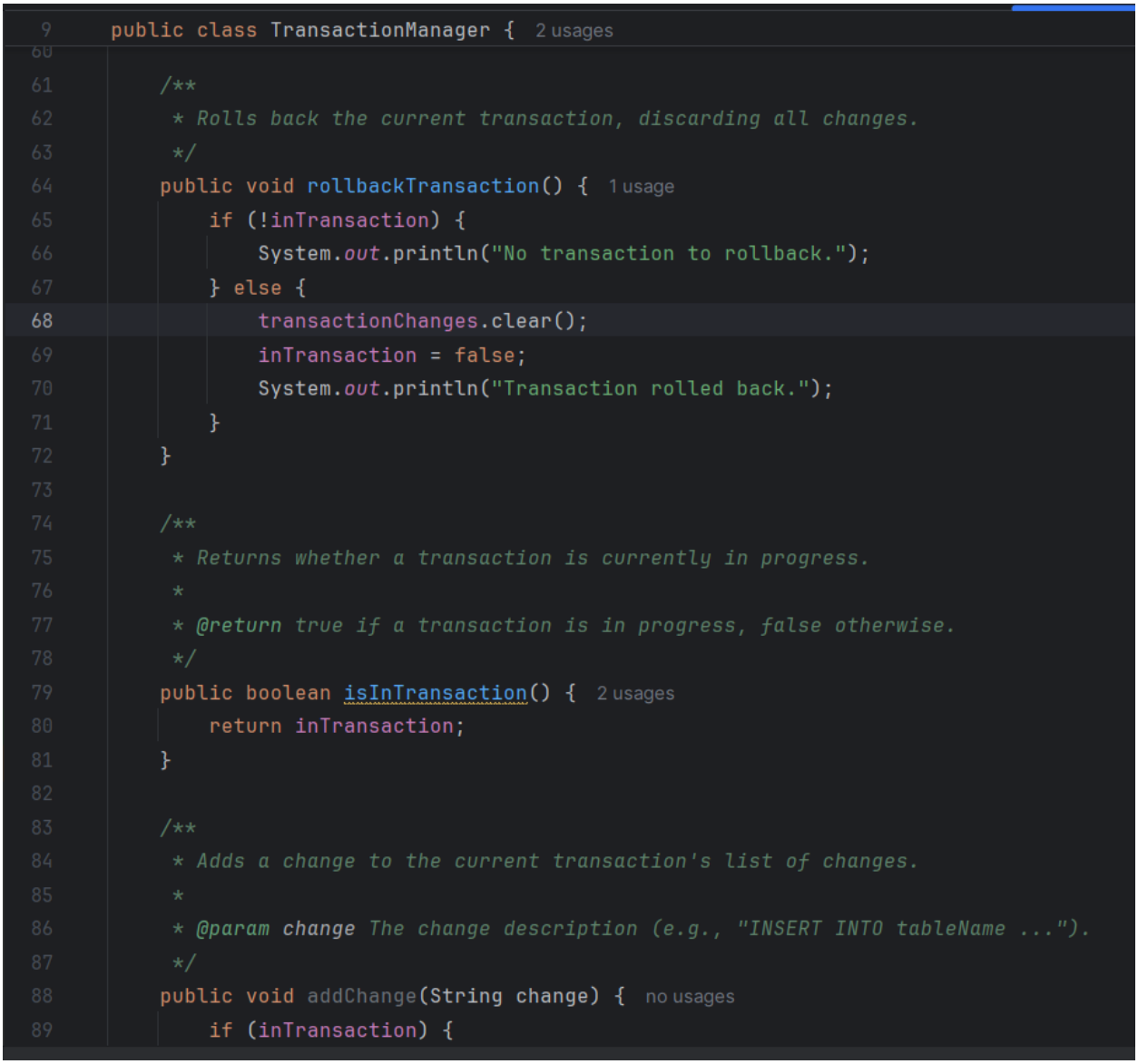
13) Task C1

Implementation of TransactionManagar



```
public class TransactionManager {  2 usages
     *
     * @param storage The persistent storage object.
     */
    public TransactionManager(PersistentStorage storage) {  1 usage
        this.storage = storage;
        this.transactionChanges = new ArrayList<>();
        this.inTransaction = false;
    }

    /**
     * Begins a new transaction.
     */
    public void beginTransaction() {  1 usage
        if (inTransaction) {
            System.out.println("A transaction is already in progress.");
        } else {
            inTransaction = true;
            transactionChanges.clear();
            System.out.println("Transaction started.");
        }
    }

    /**
     * Commits the current transaction, applying all changes.
     */
    public void commitTransaction() {  1 usage
        if (!inTransaction) {
            System.out.println("No transaction to commit.");
        } else {
            if (transactionChanges.isEmpty()) {
```

Figure 28

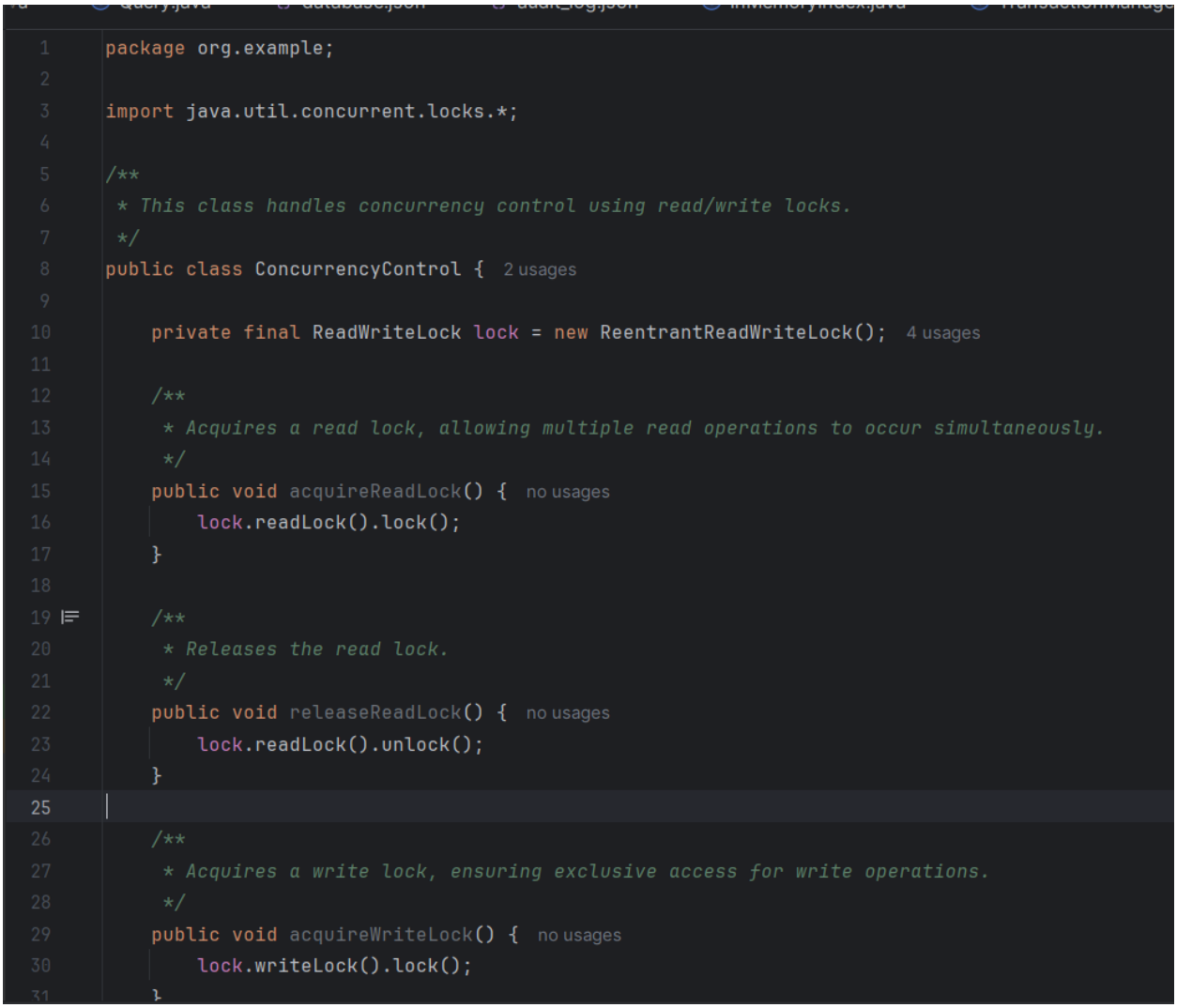


```
9   public class TransactionManager {  2 usages
60
61      /**
62       * Rolls back the current transaction, discarding all changes.
63       */
64      public void rollbackTransaction() {  1 usage
65          if (!inTransaction) {
66              System.out.println("No transaction to rollback.");
67          } else {
68              transactionChanges.clear();
69              inTransaction = false;
70              System.out.println("Transaction rolled back.");
71          }
72      }
73
74      /**
75       * Returns whether a transaction is currently in progress.
76       *
77       * @return true if a transaction is in progress, false otherwise.
78       */
79      public boolean isInTransaction() {  2 usages
80          return inTransaction;
81      }
82
83      /**
84       * Adds a change to the current transaction's list of changes.
85       *
86       * @param change The change description (e.g., "INSERT INTO tableName ...").
87       */
88      public void addChange(String change) {  no usages
89          if (inTransaction) {
```

Figure 29

14) Task C2

Implementation of ConcurrencyControl



```java
package org.example;

import java.util.concurrent.locks.*;

/**
 * This class handles concurrency control using read/write locks.
 */
public class ConcurrencyControl {  2 usages

    private final ReadWriteLock lock = new ReentrantReadWriteLock();  4 usages

    /**
     * Acquires a read lock, allowing multiple read operations to occur simultaneously.
     */
    public void acquireReadLock() {  no usages
        lock.readLock().lock();
    }

    /**
     * Releases the read lock.
     */
    public void releaseReadLock() {  no usages
        lock.readLock().unlock();
    }

    /**
     * Acquires a write lock, ensuring exclusive access for write operations.
     */
    public void acquireWriteLock() {  no usages
        lock.writeLock().lock();
    }
```

Figure 30

15) Test Cases

1) **Show Database Failed:** Without having any database it will show no database found.

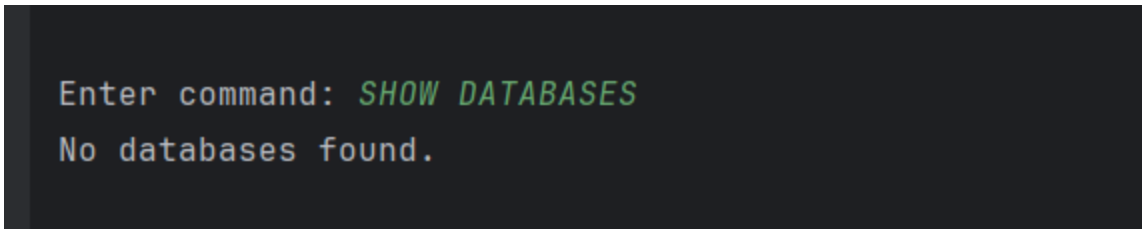


Figure 31

2) **Database Not Found –** If there is no database created it will show no database found.

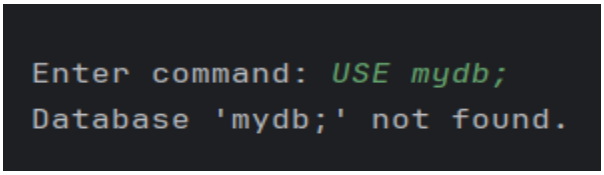


Figure 32

3) Database Successfully created: Now the database is created, and it will show the database

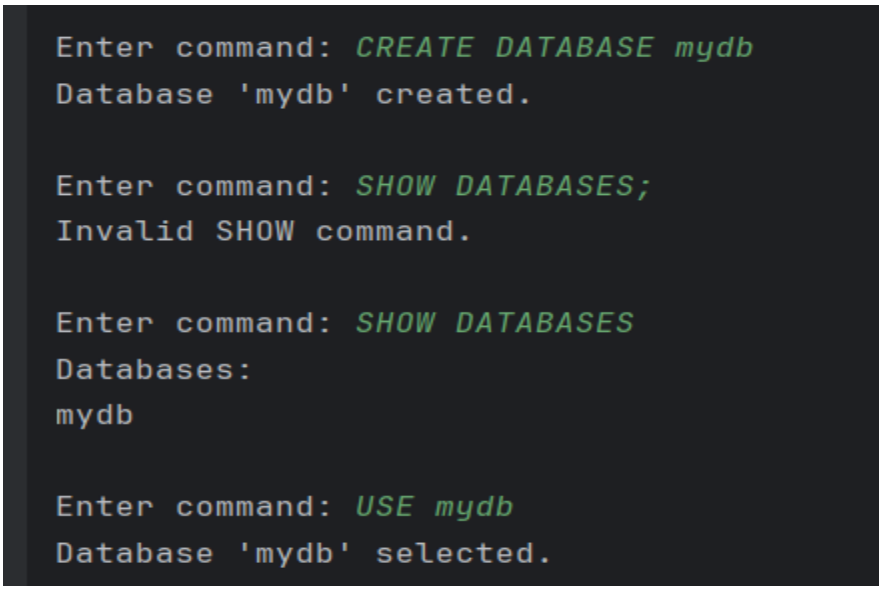


Figure 33

4) No Database selected: If a user wants to create table but they have not selected the database it shows the error.

```
Enter command: SHOW DATABASES
Databases:
mydb

Enter command: CREATE TABLE users (id INT, name STRING);
No database selected. Use USE <dbname> to select a database.

Enter command: USE mydb
Database 'mydb' selected.

Enter command: CREATE TABLE users (id INT, name STRING);
Table 'users' created in database 'mydb'.
```

Figure 34

5) Created Table in Database and failed to create a table with invalid syntax: The table is created in database, and it will failed for the duplicate names.

```
Enter command: DESCRIBE users
Columns for table 'users':
id INT
name STRING

Enter command: CREATE TABLE users (id INT, email STRING);
Table 'users' already exists.

Enter command: CREATE TABLE orders id INT, product STRING);
Syntax error in CREATE TABLE statement.
```

Figure 35

6) Insert tables and it will fail when there is a missing column: The data will inserted into the table and failed if there is a missing column.

```
Enter command: INSERT INTO users VALUES (1, "Parva")
Inserted values: [1,  "Parva"]

Enter command: INSERT INTO users VALUES (2, "Param")
Inserted values: [2,  "Param"]


 Please enter a valid command.

Enter command: INSERT INTO users VALUES (3);
Invalid number of values. Expected 2 values for table 'users'.

Enter command: INSERT INTO orders VALUES (1, 'Laptop');
Table 'orders' not found.
```

Figure 36

7) Syntax error for Select statement: It will show the syntax error for the table.

```
Enter command: SELECT users WHERE id = 1;
Invalid SELECT syntax. Use: SELECT * FROM tableName
```

Figure 37

8) Insert new data: Update the index after data insertion.

```
Enter command: INSERT INTO users VALUES (3, 'Alice');
Inserted values: [3,  'Alice';]
```

Figure 38

9) Transactions Successful: Perform operations in the transaction.

```
Enter command: BEGIN TRANSACTION
Transaction started.
Transaction started.

Enter command: INSERT INTO users VALUES (4, "MARK");
Inserted values: [4,  "MARK";]

Enter command: COMMIT;
Unknown command. Please try again.

Enter command: COMMIT
No changes to commit.
Transaction committed.

Enter command: ROLLBACK
Transaction rolled back.
```

Figure 39

10) Concurrency transactions: Resolve transactions conflicts.

```
Enter command: BEGIN TRANSACTION AS User1;
Transaction started.
Transaction started.

Enter command: BEGIN TRANSACTION as User2;
A transaction is already in progress.
```

Figure 40

16) References

1) GeeksforGeeks, "SOLID Principles in Java," *GeeksforGeeks* [Online]. Available: https://www.geeksforgeeks.org/solid-principles-java. [Accessed: March 07, 2025].

2) Draw.io, "How to Create an ER Diagram Using Draw.io," *Draw.io* [Online]. Available: https://www.draw.io. [Accessed: March 07, 2025].

3) IntelliJ IDEA, "JetBrains IntelliJ IDEA: The Java IDE for Professional Developers," *JetBrains* [Online]. Available: https://www.jetbrains.com/idea. [Accessed: March 07, 2025].

4) Oracle, "JavaDocs: Writing API Documentation in Java," *Oracle* [Online]. Available: https://docs.oracle.com/javase/8/docs/technotes/tools/windows/javadoc.html. [Accessed: March 07, 2025].

5) Lucidchart, "Create UML Diagrams with Lucidchart," *Lucidchart* [Online]. Available: https://www.lucidchart.com. [Accessed: March 07, 2025].